Failure Modes and Safety Controls for Retrieval Augmented Generation Systems

Retrieval improves evidence access, not behavioral control

Rogel S.J. Corral
Independent Researcher

March 2026
License: Licensed under CC BY-SA 4.0
https://creativecommons.org/licenses/by-sa/4.0/

Abstract

Retrieval Augmented Generation (RAG) is frequently presented in enterprise IT as a practical cure for hallucinations. This view is incomplete. RAG increases the probability that a model sees relevant material, but it does not guarantee correct interpretation, constraint adherence, policy compliance, resistance to indirect prompt injection, or stable behavior under uncertainty. In some deployments, RAG can increase operational risk when citations are interpreted as verification and when the retrieval corpus is treated as a trusted instruction channel rather than an untrusted evidence channel. This paper defines the boundary between evidence supply and behavioral control, presents a failure mode taxonomy for enterprise RAG deployments, and grounds the risks in verifiable public incidents and peer reviewed research on indirect prompt injection against retrieval based LLM systems.

Keywords: retrieval augmented generation, RAG, reliability, safety, indirect prompt injection, governance, enterprise AI.

1 Introduction

Retrieval Augmented Generation (RAG) is often introduced as the practical answer to hallucinations. The argument is intuitive. If the model has access to the right documents, it should stop guessing.

The problem is that reliability in enterprise systems is not only a knowledge availability problem. Reliability is a behavioral control problem. A model can be shown the correct paragraph and still produce the wrong conclusion, violate policy, or take an unsafe action. RAG improves evidence access. It does not enforce what the system is allowed to do with that evidence.

This position paper makes a narrow distinction. RAG improves access to relevant evidence, but it does not by itself provide enforcement, correctness guarantees, or policy compliance.

1.1 Contributions

This paper makes four contributions.

- It defines a concrete boundary for enterprise deployments: retrieval supplies evidence, while reliability and safety require enforcement.
- It presents a failure mode taxonomy that persists even when retrieval returns relevant content.
- It explains why RAG can inflate the credibility of wrong outputs and broaden the attack surface via corpus level injection.
- It grounds the claims with verifiable public incidents and peer reviewed research on indirect prompt injection against retrieval based LLM systems.

1.2 Scope and non claims

This paper does not claim that RAG is ineffective, nor that retrieval cannot improve factuality. It claims that retrieval alone does not guarantee correct interpretation, compliance, or safe behavior under pressure. The focus is enterprise usage where RAG supports operational decisions, internal knowledge access, and workflows that may influence high impact actions. This paper is defensive and does not provide offensive code. The intent is to clarify boundary conditions and risk classes, not to critique teams adopting RAG as a practical grounding technique.

2 Failure mode taxonomy

2.1 Failure Mode A: RAG solves the evidence problem, not the behavior problem

A model without retrieval often hallucinates because it lacks facts. RAG can reduce that by increasing the probability that relevant passages enter the context window.

But reliability is not primarily a lack of facts problem. Reliability is a what the system will do with the facts problem.

A model can be given the correct paragraph and still produce a wrong answer because it can:

- misread a constraint, exception, or qualifier
- conflate similar concepts
- generalize beyond what the text supports
- omit conditions that make the recommendation safe
- paraphrase into a meaning the source does not contain

In short, RAG can make the model better informed and still wrong. A better informed wrong answer is often a larger risk because it is easier to trust.

2.2 Failure Mode B: RAG increases credibility even when the answer is wrong

A failure without citations is easier to challenge. A failure with citations looks legitimate.

This is an underappreciated enterprise risk. RAG can create an illusion of verification. Users see references and assume correctness. The model may have retrieved the right document and still extracted the wrong implication, or it may cite a document while contradicting its actual constraints.

In regulated or safety sensitive environments, "wrong with sources" can be worse than "wrong without sources" because it is harder to detect and more likely to be acted upon.

2.3 Failure Mode C: RAG does not eliminate prompt injection risk; it changes the attack surface

Prompt injection is often discussed as a prompt level issue. In RAG deployments, the retrieval corpus becomes an additional input channel that must be treated as untrusted by default.

If untrusted or user influenced content can enter the retrieval store, then corpus content can become a durable channel for instruction contamination, including adversarial manipulation. Peer reviewed research demonstrates indirect prompt injection against end to end RAG application frameworks and evaluates success rates across configurations [1]. Recent work emphasizes that malicious instructions can be made retrievable under natural queries and can coerce retrieval based systems into unsafe behavior [2].

Even without a malicious actor, normal corporate content contains persuasive language, procedural shortcuts, and informal guidance that can override policy intent when surfaced out of context. RAG is not only retrieval. It is a new input boundary. Boundaries must be defended.

2.4 Failure Mode D: RAG does not enforce policy compliance

RAG can retrieve a policy. It cannot guarantee adherence to it.

This is where IT teams often overestimate what retrieval accomplishes. If the model is optimizing for helpfulness and fluency, it may treat policy text as reference rather than constraint. Without explicit enforcement, "policy in context" becomes "policy as a suggestion."

Compliance requires mechanisms that can reject, repair, or route outputs before they are emitted.

2.5 Failure Mode E: RAG does not resolve ambiguity; it amplifies it unless governed

Enterprise knowledge bases are rarely clean. They contain:

- outdated versions and superseded procedures
- overlapping documents from different teams
- conflicting interpretations of the same requirement
- partial excerpts that remove critical qualifiers
- local exceptions that do not generalize

Retrieval ranks content by relevance signals, not by correctness, authority, or current validity unless engineered to do so. Without authority tiers, precedence rules, and conflict detection, the system can synthesize confident output from contradictory material. Confidence is not resolution.

2.6 Failure Mode F: RAG does not solve tail risk; tail risk is where enterprise failures live

Most demonstrations look good in the average case. Production failures happen in the tail:

- edge case requests
- adversarial or manipulative prompts
- incomplete inputs
- contradictory data
- time pressure

RAG reduces one tail condition, missing knowledge. It does not address the more dangerous tail condition: unsafe behavior when uncertainty is high. When the correct response is insufficient evidence, retrieval alone does not guarantee the system will stop.

3 Real world scenarios and verifiable signals

3.1 Scenario 1: Zero click prompt injection and data exposure in a production assistant

EchoLeak is a documented prompt injection exploit affecting Microsoft 365 Copilot (CVE-2025-32711), described as enabling data exfiltration via crafted content without typical user driven prompting [3]. The relevance here is the pattern. When systems combine

untrusted content channels with LLM context assembly and access to internal data or actions, safety becomes an input boundary and enforcement problem, not a retrieval problem.

3.2 Scenario 2: Single click prompt injection chain against an enterprise assistant

Varonis Threat Labs published Reprompt, described as a single click prompt injection chain against Microsoft Copilot that could trigger unintended data exposure via crafted link parameters, later patched [4]. Independent security coverage summarizes the same exploit and mitigation timeline [5].

3.3 Scenario 3: Measured success of indirect prompt injection against RAG frameworks

Rag and Roll provides an end to end evaluation of indirect prompt manipulations in LLM based application frameworks, reporting that attacks can succeed at material rates across configurations [1]. This supports a practical engineering conclusion. Security is not implied by retrieval. It must be engineered into the pipeline.

3.4 Scenario 4: Indirect prompt injection made retrievable under natural queries

Overcoming the Retrieval Barrier presents indirect prompt injection exploits under natural queries spanning RAG and agentic systems, emphasizing that retrieval is a critical vulnerability when malicious instructions can be made reliably retrievable [2].

3.5 Boundary condition: retrieved text is evidence, not authority

Retrieved text should be treated as untrusted reference material by default, even when it comes from internal systems. Retrieval improves evidence access, but it does not confer authority on the retrieved content.

A RAG pipeline therefore requires explicit instruction hierarchy. System constraints and approved policy must remain binding. Retrieved passages should inform an answer only within those constraints, and conflicts should trigger refusal, escalation, or a request for clarification rather than improvised synthesis.

4 What a safety system must add beyond retrieval

RAG is a component. Reliability is an architecture.

In practice, enterprise reliability requires a governed runtime path that can:

- validate outputs against hard constraints
- detect missing evidence and refuse or escalate
- enforce policy boundaries even under adversarial pressure
- apply structured repair rather than improvised rewriting
- produce auditable traces of why an output was allowed

Retrieval can feed such a system. Retrieval cannot replace it.

4.1 A practical test

When the retriever returns an outdated procedure, a plausible but irrelevant chunk, a conflict between authoritative sources, or instruction like content embedded in an untrusted document, the system must specify what happens next.

If the operational assumption is that the model will reliably resolve these conditions on its own, then the pipeline lacks enforceable control boundaries. In that case, correctness and compliance become emergent properties of model behavior rather than designed guarantees enforced by the system.

5 Conclusion

RAG is valuable. In many enterprise contexts it is the correct first step for grounding, provided it is paired with explicit instruction precedence and output enforcement. It can improve factuality and reduce a common class of hallucination caused by missing information. But it is not an ultimate solution because it does not guarantee correct interpretation, policy compliance, adversarial robustness, or stable behavior in high uncertainty conditions.

RAG answers the question: can the model see relevant information. Reliability answers the question: will the system behave correctly even when pressured, confused, or attacked.

IT has seen this pattern before. Tools improve capability. Engineering provides guarantees. When organizations confuse the two, the failure is not surprising. It is scheduled.

References

[1] G. De Stefano, L. Schönherr, and G. Pellegrino. Rag and Roll: An End-to-End Evaluation of Indirect Prompt Manipulations in LLM-based Application Frameworks. arXiv:2408.05025 (2024). arXiv:2408.05025

[2] H. Chang, E. Bao, X. Luo, and T. Yu. Overcoming the Retrieval Barrier: Indirect Prompt Injection in the Wild for LLM Systems. arXiv:2601.07072 (2026). arXiv:2601.07072

[3] P. Reddy and A. S. Gujral. EchoLeak: The First Real-World Zero-Click Prompt Injection Exploit in a Production LLM System. arXiv:2509.10540 (2025). arXiv:2509.10540

[4] Varonis Threat Labs. Reprompt: The Single-Click Microsoft Copilot Attack that Silently Steals Your Personal Data. January 26, 2026. Varonis (Reprompt)

[5] P. Arntz. Reprompt attack lets attackers steal data from Microsoft Copilot. Malwarebytes, January 15, 2026. Malwarebytes (Reprompt)